

TEORIA DA COMPUTAÇÃO

Teoria da Complexidade e Análise de Tempo de Algoritmos

Subconjunto Soma (Subset Sum)

Análise teórica e experimental de uma solução por backtracking

Disciplina	Teoria da Computação
Professor	Daniel Bezerra
Equipe	Davi Marques Oliveira, Pedro Henrique Martins, João Lucas Veiga, Felipe Menelau
Linguagens	Python e Java
Repositório	https://github.com/davimqz/Projeto-Teoria
Data	Junho de 2026

Sumário

Resumo	3
1. Introdução	3
2. Definição formal do problema	3
3. Descrição do algoritmo	4
3.1 Estratégia de backtracking.....	4
3.2 Pseudocódigo	4
4. Implementação.....	4
5. Análise de complexidade	5
6. Metodologia experimental	6
7. Resultados experimentais.....	7
8. Gráficos	8
9. Discussão dos resultados	11
10. Aplicabilidade e limitações do algoritmo	12
11. Relação com P, NP e NP-completude	12
12. Conclusão.....	13
13. Referências	13

Resumo

Este relatório apresenta a implementação e a análise do problema Subconjunto Soma, também conhecido como Subset Sum. Dado um conjunto de inteiros positivos e um valor-alvo W , a versão de decisão do problema pergunta se existe algum subconjunto cujos elementos somem exatamente W . Apesar do enunciado simples, esse é um problema central em Teoria da Computação, pois sua versão de decisão é NP-completa.

Para resolvê-lo, escolhemos a estratégia de backtracking recursivo com poda para entradas positivas. A cada nível da recursão, o algoritmo decide incluir ou excluir o elemento atual, formando uma árvore binária de decisões. No pior caso, a busca percorre essa árvore por completo, o que leva a um crescimento exponencial da ordem de $O(2^n)$.

A parte experimental foi desenvolvida em duas linguagens distintas, Python e Java, usando exatamente as mesmas instâncias de entrada para garantir uma comparação justa. Geramos 720 instâncias por linguagem, distribuídas entre os cenários de melhor, médio e pior caso, com 30 rodadas para cada combinação de cenário e tamanho. Ao final, os dois programas produziram a mesma resposta e o mesmo número de chamadas recursivas em todas as instâncias, o que reforça que a diferença observada entre elas é apenas de desempenho, e não de estratégia algorítmica.

Palavras-chave: Subset Sum; Subconjunto Soma; Backtracking; Complexidade Computacional; NP-completude; Análise de Algoritmos.

1. Introdução

A análise de algoritmos nos permite estudar como o tempo de execução de um programa cresce em função do tamanho da entrada. Esse estudo é especialmente importante em problemas combinatórios, nos quais o número de possibilidades a serem examinadas pode explodir rapidamente à medida que a entrada aumenta. O Subset Sum é um exemplo clássico desse tipo de comportamento.

Formalmente, o Subset Sum é um problema de decisão: dado um conjunto S de n inteiros positivos e um alvo W , queremos saber se existe um subconjunto de S cuja soma seja igual a W . A pergunta parece inofensiva, mas resolvê-la para qualquer entrada possível é, até onde se sabe, um problema difícil. Foi justamente esse contraste entre a simplicidade do enunciado e a dificuldade real que nos motivou a escolher o backtracking como abordagem de estudo.

Optamos pelo backtracking porque ele torna a explosão combinatória bastante visível. Para cada elemento do conjunto, a busca se ramifica em duas alternativas, incluir ou excluir, e esse padrão se repete em todos os níveis. Comparando uma implementação em Python e outra em Java sobre as mesmas entradas, conseguimos observar tanto o comportamento assintótico previsto pela teoria quanto as diferenças práticas entre as duas linguagens.

2. Definição formal do problema

Entrada: um vetor $S = \{a_1, a_2, \dots, a_n\}$ de inteiros positivos e um valor-alvo W .

Saída: verdadeiro, se existe um subconjunto $S' \subseteq S$ cuja soma dos elementos seja igual a W ; falso, caso contrário.

Componente	Descrição
S	Lista (ou vetor) com n inteiros positivos.
W	Valor-alvo a ser atingido por algum subconjunto.
Saída verdadeira	Existe pelo menos um subconjunto cuja soma é exatamente W .
Saída falsa	Nenhum subconjunto soma exatamente W .

Como exemplo, considere $S = \{3, 34, 4, 12, 5, 2\}$ e $W = 9$. A resposta é verdadeira, pois o subconjunto $\{4, 5\}$ soma exatamente 9.

3. Descrição do algoritmo

3.1 Estratégia de backtracking

O algoritmo percorre uma árvore de decisão de profundidade n . Em cada nível, que corresponde a um elemento do vetor, fazemos duas chamadas recursivas: uma que inclui o elemento na soma parcial e outra que o ignora. A busca retorna verdadeiro assim que a soma parcial se iguala a W . Se todos os elementos forem examinados sem atingir W , ela retorna falso.

Como as entradas usadas nos experimentos contêm apenas inteiros positivos, podemos aplicar a poda $somaAtual > W$ com segurança. Essa poda encerra um ramo da busca quando a soma parcial já ultrapassou o alvo, pois, sem números negativos, não há como uma soma que excedeu W voltar a igualá-lo. Vale registrar que essa poda não seria correta se números negativos fossem permitidos.

3.2 Pseudocódigo

```

Algoritmo SubsetSumBacktracking( $S, W$ )
    retorne Buscar( $S, W, 0, 0$ )

Função Buscar( $S, W, i, somaAtual$ )
    se  $somaAtual == W$  então
        retorne verdadeiro
    se  $i == tamanho(S)$  então
        retorne falso
    se  $somaAtual > W$  então
        retorne falso           // poda válida para inteiros positivos
    se Buscar( $S, W, i + 1, somaAtual + S[i]$ ) então
        retorne verdadeiro
    retorne Buscar( $S, W, i + 1, somaAtual$ )

```

4. Implementação

Mantivemos as duas implementações o mais próximas possível, com a mesma ordem de exploração da árvore de decisão e a mesma poda para inteiros positivos. A intenção foi isolar o efeito da linguagem: como o algoritmo é idêntico, qualquer diferença de tempo reflete características de execução de cada ambiente, e não escolhas diferentes de implementação.

Aspecto	Python	Java
Arquivo do algoritmo	python/subset_sum.py	java/src/SubsetSum.java
Arquivo de benchmark	python/benchmark.py	java/src/Benchmark.java
Função de tempo	time.perf_counter()	System.nanoTime()
Contador de chamadas	recursive_calls no CSV	recursive_calls no CSV
Entrada compartilhada	data/instances.csv	data/instances.csv
Poda	current_sum > target	currentSum > target

Após executar os benchmarks, conferimos os resultados das duas linguagens instância por instância. Em todas as 720 instâncias, Python e Java retornaram o mesmo valor booleano e o mesmo número de chamadas recursivas: zero divergências de resposta e zero divergências de contagem. Essa coincidência total é um bom indício de que as duas implementações realmente percorrem a árvore de decisão da mesma maneira.

5. Análise de complexidade

5.1 Pior caso — Big-O

No pior caso, nenhuma solução é encontrada e a poda não elimina ramos relevantes, de modo que a busca precisa visitar toda a árvore de decisão. Uma árvore binária de profundidade n possui até $2^{n+1} - 1$ nós, o que corresponde a um crescimento assintótico $O(2^n)$. A recorrência que descreve esse comportamento é $T(n) = 2 \cdot T(n-1) + O(1)$, cuja solução é exponencial.

5.2 Melhor caso — Big-Ω

O melhor caso ocorre quando a solução aparece logo nas primeiras chamadas. Nas instâncias que construímos para esse cenário, o primeiro elemento do vetor é igual ao alvo, então a busca encontra a resposta após apenas duas chamadas recursivas: a chamada inicial e a chamada que inclui o primeiro elemento. Como isso independe de n , o comportamento é constante, $\Omega(1)$.

5.3 Caso médio

O caso médio depende da distribuição dos valores, do alvo e da posição em que uma eventual solução se encontra na árvore. Por isso, em vez de tentar uma fórmula fechada, tratamos esse cenário de forma experimental, com entradas aleatórias, alvo entre 40% e 60% da soma total dos elementos e 30 rodadas por tamanho.

5.4 Big-Θ e espaço

Para instâncias que exigem a exploração extensa da árvore, o tempo é $\Theta(2^n)$, já que o número de nós visitados acompanha o tamanho da árvore de decisão. Quanto ao espaço adicional, ele é $O(n)$, pois a profundidade máxima da pilha de recursão é igual ao número de elementos do conjunto.

Aspecto	Classificação	Justificativa
Melhor caso	$\Omega(1)$	Solução encontrada nas primeiras chamadas.
Pior caso	$O(2^n)$	Exploração de até 2^n subconjuntos.

Aspecto	Classificação	Justificativa
Exploração extensa	$\Theta(2^n)$	Crescimento proporcional à árvore de decisão.
Espaço	$O(n)$	Pilha recursiva com profundidade máxima n .

6. Metodologia experimental

6.1 Ambiente de execução

Os benchmarks foram executados em um único computador, com os programas em segundo plano minimizados para reduzir interferências nas medições. A tabela abaixo descreve o ambiente utilizado.

Componente	Valor
Processador	8GB
Memória RAM	intel core i5 14450hx 2.4
Sistema operacional	Windows 11 Pro
Versão do Python	Python 3.14
Versão do Java	Java 21
Modo de execução	Terminal do VsCode, com programas em segundo plano minimizados.

6.2 Geração das entradas

Um detalhe importante da metodologia é que as entradas foram geradas uma única vez, com semente fixa, e reutilizadas pelas duas linguagens. Isso evita o erro de comparar Python e Java sobre instâncias diferentes, que tornaria a comparação injusta. O script `data/generate_instances.py` é responsável por essa geração.

Cenário	Construção	Comportamento esperado
Melhor caso	Primeiro elemento igual ao alvo ($W = 1000$).	Retorno quase imediato, 2 chamadas recursivas.
Caso médio	Vetor aleatório e alvo entre 40% e 60% da soma total.	Tempo variável; a solução pode aparecer cedo ou tarde.
Pior caso	$W = \text{soma}(S) + 1$.	Não há solução e a poda nunca é ativada.

6.3 Tamanhos e rodadas

Trabalhamos com três tamanhos principais — pequeno, médio e grande — e acrescentamos tamanhos intermediários para deixar as curvas dos gráficos mais suaves. Cada combinação de cenário e tamanho foi executada 30 vezes em cada linguagem, o que permite calcular média e desvio-padrão de forma confiável.

Item	Valor
Tamanhos principais	$n = 10, 15$ e 20 .
Tamanhos adicionais	$n = 8, 12, 14, 16$ e 18 .
Rodadas por cenário e tamanho	30 rodadas.
Total de instâncias	720.
Total de medições (Python / Java)	720 / 720.
Total de linhas agregadas	48.

6.4 Validação lógica dos resultados

Antes de analisar os tempos, validamos a corretude do pior caso pelo número de chamadas recursivas. Como nesse cenário a árvore é percorrida por completo, o número de chamadas deve coincidir exatamente com $2^{(n+1)} - 1$. A tabela a seguir mostra que as chamadas médias observadas nos resultados batem com o valor teórico para todos os tamanhos, sem nenhum desvio.

n	Chamadas observadas	$2^{(n+1)} - 1$ esperado	Status
8	511	511	OK
10	2047	2047	OK
12	8191	8191	OK
14	32767	32767	OK
15	65535	65535	OK
16	131071	131071	OK
18	524287	524287	OK
20	2097151	2097151	OK

7. Resultados experimentais

As tabelas a seguir resumem os resultados, com os valores de tempo em milissegundos. Cada linha representa a média de 30 rodadas para uma combinação de linguagem, cenário e tamanho de entrada. Os dados foram extraídos do arquivo `results/summary_results.csv`, gerado a partir das medições brutas das duas linguagens.

7.1 Melhor caso

Linguagem	n	Rodadas	Média (ms)	Desvio	Mínimo	Máximo	Chamadas méd.
Java	8	30	0.002260	0.005677	0.000100	0.023500	2.00
Java	10	30	0.000150	0.000057	0.000100	0.000300	2.00
Java	12	30	0.000630	0.002375	0.000100	0.013200	2.00
Java	14	30	0.000167	0.000071	0.000099	0.000400	2.00
Java	15	30	0.000173	0.000069	0.000100	0.000400	2.00
Java	16	30	0.000197	0.000103	0.000100	0.000500	2.00
Java	18	30	0.000180	0.000066	0.000100	0.000300	2.00
Java	20	30	0.000207	0.000139	0.000100	0.000700	2.00
Python	8	30	0.001860	0.002590	0.000800	0.014800	2.00
Python	10	30	0.001003	0.000100	0.000900	0.001200	2.00
Python	12	30	0.003540	0.014479	0.000800	0.080200	2.00
Python	14	30	0.000903	0.000061	0.000800	0.001100	2.00
Python	15	30	0.000910	0.000066	0.000800	0.001000	2.00
Python	16	30	0.001513	0.002820	0.000800	0.016400	2.00
Python	18	30	0.000943	0.000143	0.000800	0.001600	2.00
Python	20	30	0.001723	0.004361	0.000800	0.024800	2.00

7.2 Caso médio

Linguagem	n	Rodadas	Média (ms)	Desvio	Mínimo	Máximo	Chamadas méd.
Java	8	30	0.002037	0.001948	0.000400	0.011300	357.17
Java	10	30	0.004190	0.001759	0.000400	0.006201	1168.13
Java	12	30	0.008680	0.006639	0.000200	0.022200	2506.30
Java	14	30	0.010737	0.011631	0.000100	0.038600	3027.23
Java	15	30	0.009870	0.008288	0.000500	0.028599	2502.77
Java	16	30	0.009557	0.008730	0.000300	0.035901	2681.87
Java	18	30	0.006383	0.005521	0.000400	0.021700	2188.40
Java	20	30	0.004523	0.006920	0.000200	0.034199	2303.30
Python	8	30	0.032577	0.009448	0.002300	0.057700	357.17
Python	10	30	0.118283	0.066080	0.005100	0.283900	1168.13
Python	12	30	0.230377	0.190281	0.001800	0.615900	2506.30
Python	14	30	0.287113	0.342307	0.001700	1.225800	3027.23
Python	15	30	0.243790	0.230584	0.007900	0.823600	2502.77
Python	16	30	0.275873	0.312455	0.004800	1.489400	2681.87
Python	18	30	0.206590	0.203295	0.007000	0.763700	2188.40
Python	20	30	0.213070	0.357869	0.002800	1.724100	2303.30

7.3 Pior caso

Linguagem	n	Rodadas	Média (ms)	Desvio	Mínimo	Máximo	Chamadas méd.
Java	8	30	0.000786	0.000057	0.000699	0.000999	511.00
Java	10	30	0.002800	0.000059	0.002699	0.002900	2047.00
Java	12	30	0.011377	0.000077	0.011299	0.011600	8191.00
Java	14	30	0.044164	0.000156	0.043801	0.044500	32767.00
Java	15	30	0.068516	0.002639	0.065399	0.079500	65535.00
Java	16	30	0.188833	0.028087	0.176100	0.309301	131071.00
Java	18	30	0.713440	0.014122	0.698200	0.764800	524287.00
Java	20	30	3.137550	1.086273	2.805800	8.780900	2097151.00
Python	8	30	0.124420	0.419135	0.043700	2.343000	511.00
Python	10	30	0.183987	0.013410	0.173000	0.226600	2047.00
Python	12	30	0.737770	0.043720	0.683900	0.897200	8191.00
Python	14	30	3.104837	0.339015	2.815600	4.570500	32767.00
Python	15	30	6.271440	0.472564	5.700300	7.735700	65535.00
Python	16	30	12.415390	0.541835	11.523300	13.657700	131071.00
Python	18	30	49.896877	1.311458	47.042000	53.395600	524287.00
Python	20	30	201.960813	6.197826	194.301400	219.889500	2097151.00

8. Gráficos

Os gráficos foram gerados pelo script `analysis/generate_graphs.py`. O eixo X representa o tamanho da entrada `n` e o eixo Y representa o tempo médio em milissegundos. Nos gráficos do pior caso usamos escala

logarítmica no eixo Y e sobrepomos a curva teórica proporcional a 2^n sobre os dados medidos, de modo a evidenciar a aderência dos experimentos à previsão teórica.

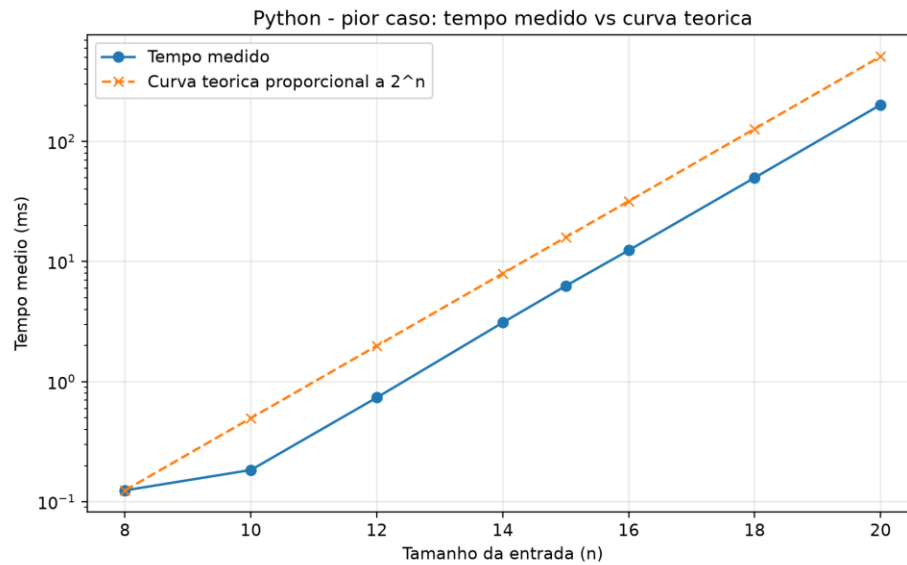


Figura 1 — Python no pior caso: tempo medido versus curva teórica proporcional a 2^n .

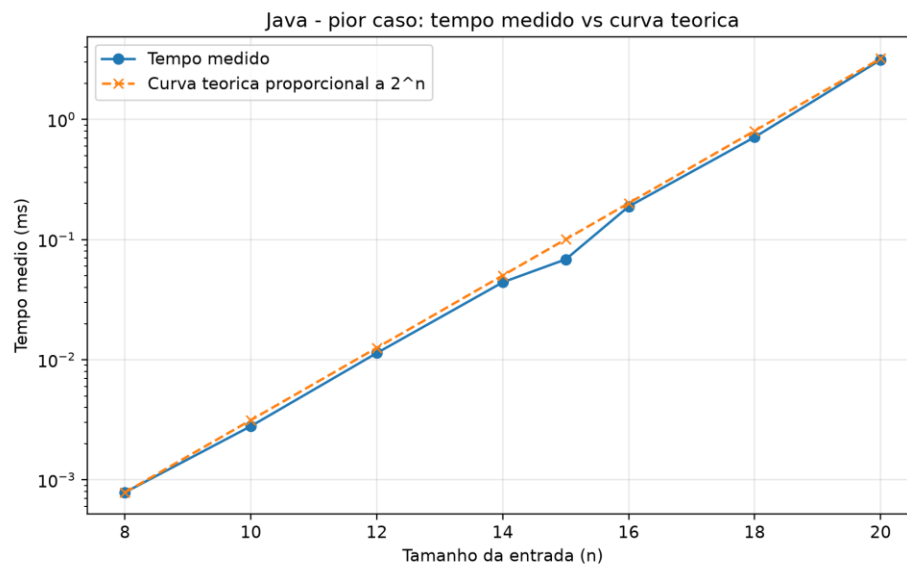


Figura 2 — Java no pior caso: tempo medido versus curva teórica proporcional a 2^n .

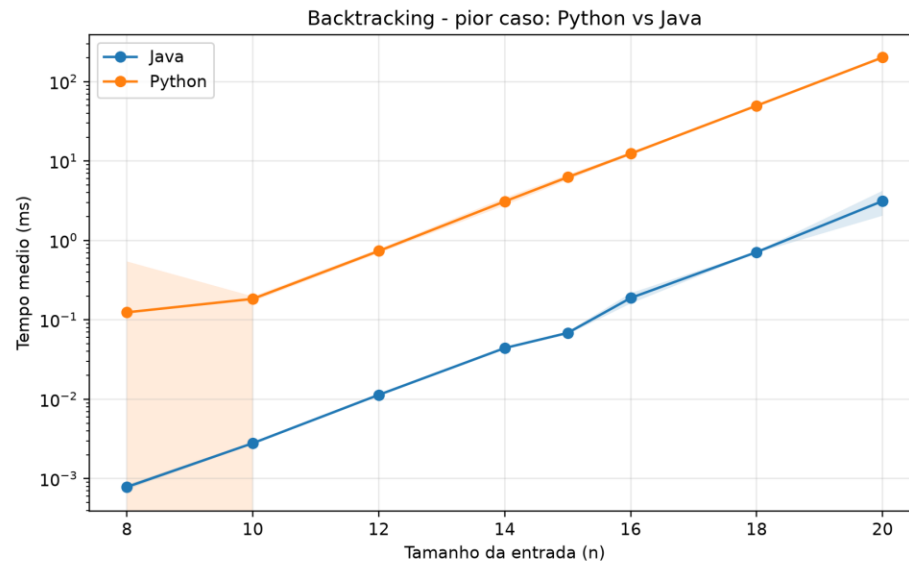


Figura 3 — Comparação entre Python e Java no pior caso.

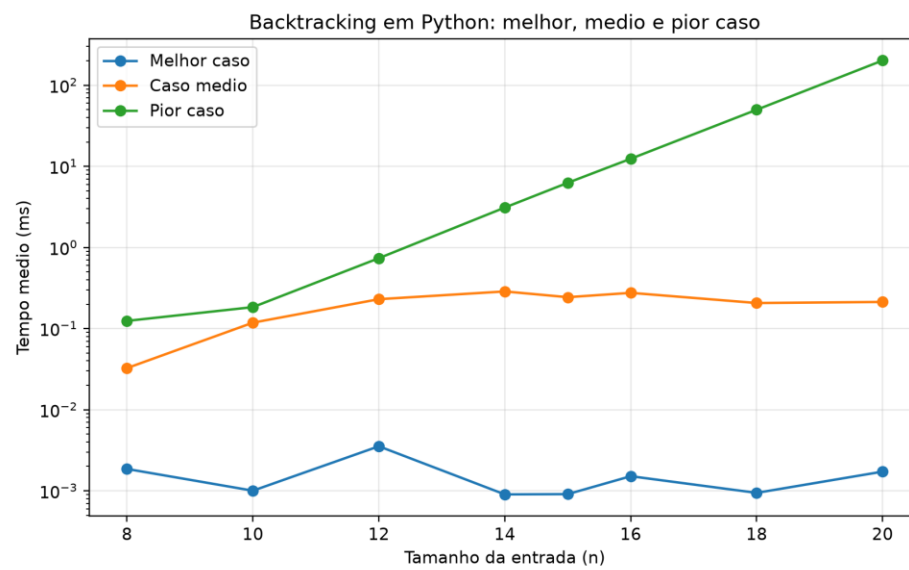


Figura 4 — Python: melhor caso, caso médio e pior caso.

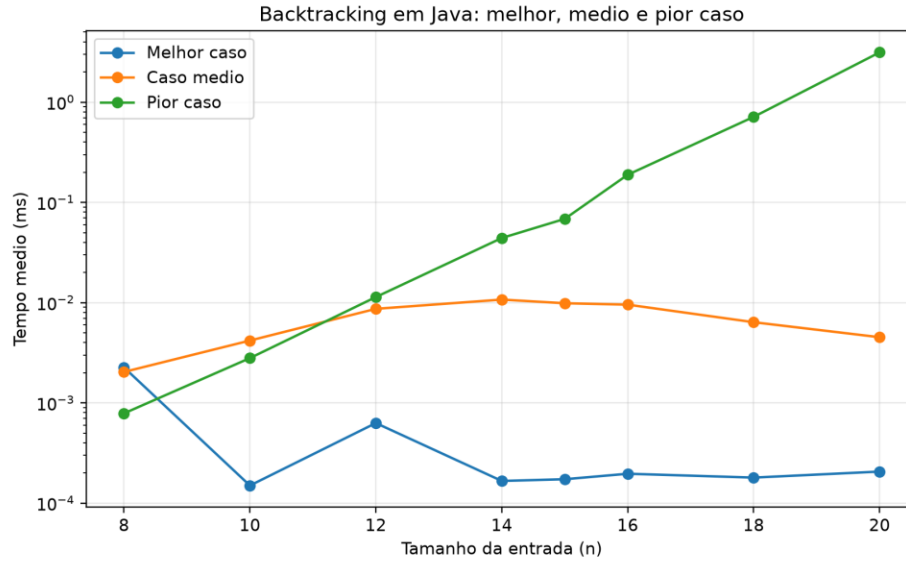


Figura 5 — Java: melhor caso, caso médio e pior caso.

9. Discussão dos resultados

9.1 Aderência à complexidade teórica

O pior caso se comportou de forma compatível com a análise $O(2^n)$. O número médio de chamadas recursivas foi exatamente $2^{(n+1)} - 1$ para cada tamanho, o que indica que a árvore de decisão foi visitada por inteiro e confirma que a construção $W = \text{soma}(S) + 1$ realmente força o pior caso. Em termos de tempo, no Python o pior caso passou de cerca de 0,184 ms em $n = 10$ para aproximadamente 201,96 ms em $n = 20$; no Java, passou de cerca de 0,0028 ms para 3,14 ms no mesmo intervalo. A diferença absoluta entre as linguagens não altera a classe de complexidade: ela reflete constantes de execução, otimizações da JVM e o overhead do interpretador do Python.

9.2 Comparação entre Python e Java

No pior caso, o Java foi consideravelmente mais rápido. A tabela abaixo mostra a razão entre os tempos médios das duas linguagens para alguns tamanhos. Mesmo com essa diferença de desempenho, ambas executaram o mesmo número de chamadas recursivas, o que confirma que a vantagem do Java vem da implementação e do ambiente de execução, e não de uma estratégia algorítmica distinta.

n	Python — pior caso (ms)	Java — pior caso (ms)	Razão Python/Java
10	0.183987	0.002800	65.71×
15	6.271440	0.068516	91.53×
20	201.960813	3.137550	64.37×

9.3 Melhor caso e caso médio

No melhor caso, o número médio de chamadas permaneceu igual a 2 em todos os tamanhos, o que confirma o comportamento constante previsto. Já no caso médio, os tempos não crescem de forma

monótona com n . Isso é esperado para o backtracking sobre instâncias aleatórias: em algumas entradas a solução aparece cedo na árvore e a busca termina rápido, enquanto em outras é preciso explorar bem mais antes de decidir. Essa variabilidade fica visível nos desvios-padrão relativamente altos do caso médio.

9.4 Limitações experimentais

- Tempos muito pequenos, sobretudo no melhor caso e no Java, sofrem maior influência do ruído do sistema operacional e da resolução de medição, o que explica alguns valores que oscilam entre tamanhos próximos.
- Os resultados refletem o ambiente específico em que foram coletados; em outra máquina, os tempos absolutos podem mudar, embora a tendência exponencial do pior caso e a relação entre as linguagens tendam a se manter.
- Os tamanhos foram limitados a $n \leq 20$ justamente porque, a partir daí, o custo do pior caso cresce rápido demais para uma coleta confortável de 30 rodadas por configuração.

10. Aplicabilidade e limitações do algoritmo

O backtracking é uma boa escolha quando n é pequeno ou moderado, quando se deseja uma solução exata de implementação simples, ou quando o objetivo é didático, como demonstrar o crescimento combinatório do espaço de busca. Ele também serve de base para estratégias mais sofisticadas, como branch and bound, memoização e meet-in-the-middle.

Sua principal limitação é a escalabilidade. Em instâncias grandes, o número de subconjuntos cresce exponencialmente, e a poda, embora ajude em alguns casos, não altera o limite assintótico do pior caso. Por isso, para entradas grandes, abordagens puramente baseadas em backtracking deixam de ser viáveis.

11. Relação com P, NP e NP-completude

11.1 Pertencimento a NP

A versão de decisão do Subset Sum pertence a NP, porque, dado um subconjunto candidato, conseguimos verificar em tempo polinomial se a soma de seus elementos é igual a W . Essa verificação exige somar até n elementos e comparar o resultado com o alvo, o que custa $O(n)$.

11.2 NP-completude

Mais do que estar em NP, o Subset Sum de decisão é NP-completo: ele está em NP e é NP-difícil. Não se conhece algoritmo polinomial capaz de resolver todos os casos gerais do problema, e a existência de um implicaria $P = NP$, uma das questões em aberto mais importantes da computação.

11.3 Observação sobre programação dinâmica

Existe uma solução por programação dinâmica com complexidade $O(n \cdot W)$. À primeira vista isso poderia sugerir uma solução eficiente, mas essa complexidade é pseudo-polinomial: ela depende do valor

numérico de W , e não apenas do tamanho da entrada medido em bits. Por isso, essa abordagem não prova que o Subset Sum geral pertença a P .

12. Conclusão

O estudo do Subset Sum por backtracking nos permitiu observar, na prática, o que a teoria prevê. A evidência mais clara veio do pior caso: para todos os tamanhos testados, o número de chamadas recursivas coincidiu exatamente com $2^{(n+1)} - 1$, validando a exploração completa da árvore de decisão. Os gráficos confirmaram a tendência exponencial esperada e tornaram visível a diferença de desempenho entre as duas linguagens.

A comparação entre Python e Java mostrou que, embora o Java tenha sido bem mais rápido em termos absolutos, as duas implementações realizaram o mesmo trabalho algorítmico, com resultados e contagens de chamadas idênticos em todas as instâncias. Isso reforça uma ideia central da disciplina: a classe de complexidade descreve o crescimento do custo com a entrada, enquanto a escolha da linguagem afeta apenas as constantes desse crescimento.

Por fim, a discussão sobre P , NP e NP -completude situou o problema em seu devido lugar: o Subset Sum é simples de enunciar e de verificar, mas difícil de resolver no caso geral. O backtracking se mostra adequado para fins didáticos e para entradas pequenas, mas evidencia, ao mesmo tempo, por que problemas NP -completos exigem técnicas mais elaboradas quando a escala aumenta.

13. Referências

- CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. Introduction to Algorithms. 4. ed. Cambridge: MIT Press, 2022.
- GAREY, M. R.; JOHNSON, D. S. Computers and Intractability: A Guide to the Theory of NP-Completeness. New York: W. H. Freeman, 1979.
- KARP, R. M. Reducibility Among Combinatorial Problems. In: Complexity of Computer Computations. New York: Springer, 1972. p. 85–103.
- KLEINBERG, J.; TARDOS, E. Algorithm Design. Boston: Pearson, 2005.
- SIPSER, M. Introduction to the Theory of Computation. 3. ed. Boston: Cengage Learning, 2012.